Agile! The Good, the Hype, and the Ugly (Bertrand Meyer)

Book Summary

Patrick Bucher

2025-12-12

Contents

This is a practical book that enables the reader to benefit from the good ideas of agile methods while staying away from the bad ones. Agile methods are a mix of horrible and great ideas. Instead of trying out Scrum, Extreme Programming, Lean Softwae, and Crystal all on one's own, this book provides a description and assessment of their underlying ideas.

First, those methods are described without the usual sermons and anecdotes. Second, the underlying ideas are scrutinized and assessed, thereby separating the useful from the harmful ones. Agile methods and their texts put three difficoulties in the reader's way:

1. *Partisanship*: While it's common for their inventors to argue in favour of their inventions, proponents of agile methods ignore the rules of rational discourse and ask the reader to join their cult, which is inappropriate for engineering problems.
2. *Intimidation*: Agile texts dismiss previous approaches to software development as outdated and their users as rigid and backwards. Who wants to argue against "agile" when all of this word's opposites have negative meanings?
3. *Extremism*: Proponents of some agile methods insist on the application of a method in its entirety, which discourages a more differentiated view on the techniques that make up a specific method.

While previous books criticizing agile methods were rather timid, this book will call out the bad ideas without undue dereference.

The first chapter is a summary of agile ideas. In the second chapter, the rhetoric of agile texts is analyzed. Chapter three is a description of traditional plan-based software development—to

which agile methods are the antidote. Chapters four to eight review agile ideas: principles, roles, practices, and artifacts.

Those chapters do not focus on individual methods, but on their many commonalities. Scrum, Lean, XP, and Crystal are then presented in chapter nine as combinations of those principles already discussed, each with their one single big idea emphasized.

Chapter ten described precautions organizations should take when adopting agile methods. Chapter eleven is the final assessment; it classifies agile ideas into three categories: the good, the hype, and the ugly.

This book is not a comprehensive guide to software development, but a critique of existing approaches. Authors usually argument by gut feeling, by experience (e.g. projects saved by agile approaches and failed by ignoring them), logical reasoning, and, ideally, empirical analysis (for which there is usually too little data available). This book relies mostly on analytical reasoning, combined with personal experience and anecdotes for illustrative purposes.

1 Overview

While agile ideas date back to the 1990s, the movement gained traction with the publication of the *Manifesto for Agile Software Development* in 2001, which was signed by many proponents of existing agile ideas. Agile software development is not a single method, but a collection of ideas grouped together to methodologies such as *Extreme Programming*, *Lean Software Development*, *Crystal*, and *Scrum*, which all select and prioritize their own subset of agile ideas. However, there are some common core characteristics to those methods:

- *Values*: general assumptions framing the agile world view
- *Principles*: organizational and technical rules based on the values
- *Roles*: responsibilities and privileges of actors involved
- *Practices*: specific activities based on the principles
- *Artifacts*: virtual and physical tools supporting the practices

1.1 Values

The fundamental assumptions of agile software development are captured in the following values:

1. *Redefined roles for developers, managers, and customers*: Some of the manager's duties are transferred to the team, such as the selection and assignment of the tasks. The developers and their code are moved into the center. Customers are no longer passive recipients of a product but active participants in the development process.

2. *No "big upfront" steps*: Activities preceding the writing of code such as gathering requirements ("the customer does not know what he wants") or creating a design ("the developers do not know what will work") are left out because they are subject to change anyway. Instead, requirements and design emerge in a continuous process involving the customer, in which the software is iteratively refactored into his acceptance.
3. *Iterative development*: Development takes place in iterations of a fixed time, usually a few weeks, for which the functionality with the highest business value is implemented by working through a prioritized list of tasks. Functionality is added iteratively.
4. *Limited, negotiated functionality*: Only the most important features, measured by their business value, will be implemented. Unused functionality is deemed wasteful and therefore not implemented in the first place. The functionality to be added is negotiated before the start of every iteration. Since it is empirically impossible to fix both scope and deadline, usually the deadline is retained, but the scope limited accordingly.
5. *Focus on quality, understood as achieved through testing*: Quality is ensured by continuous testing rather than by upfront design decisions or by sticking to development methodologies. The project's regression test suite is a central artifact and must always pass when new functionality is added.

1.2 Principles

The following principles—not the ones from the original Manifesto, but extracted from the various texts on agile software development—turn the values from above into prescriptions:

- Organizational

    1. *Put the customer at the center*: Customer representatives are involved throughout the project, which should deliver the best Return on Investment to the customer.
    2. *Let the team self-organize*: The team members pick their own tasks rather than having them assigned by a manager.
    3. *Work at a sustainable pace*: Programmers work reasonable hours rather than through intense phases with long hours to meet deadlines set by a manager.
    4. *Develop minimal software*: 1) Only essential functions are built (*minimal functionality*); 2) Only what is requested is built, and extra work for future reuse and extensibility is left out (*minimal product*); 3) Only what is delivered to the customer—programs and tests—is built (*minimal artifacts*).
    5. *Accept change*: Requirements are not complete at the beginning, but evolve as customers interact with intermediate releases. Change is considered normal.

- Technical

    1. *Develop iteratively*: Every iteration of a fixed number of weeks produces a working release, providing the customer new functionality he can try out and give feedback on, which is then considered for a later iteration. No new functionality is demanded during an ongoing iteration; such requests are taken into consideration for an upcoming iteration instead.

2. *Treat tests as a key resource*: Quality is ensured by automated tests. No new development must start until all current tests pass. A text expresses the requirements new code must satisfy and is therefore written before the production code (*test first*).
3. *Express requirements through scenarios*: A scenario (e.g. a *use case* or a *user story*) describes an interaction of a user with the system that is to be built. Scenarios are obtained from the customer and written from a user's perspective. Unlike requirements, scenarios are not complete specifications but only examples. Scenarios do not have to be collected at the beginning of the project, but can be added and modified during development.

Thus, requirements are replaced by two artifacts: tests and scenarios.